TCS302 - DATA STRUCTURES WITH C

Quick Sort

Algorithm, Implementation and Complexity

Unit IV - Lecture 4

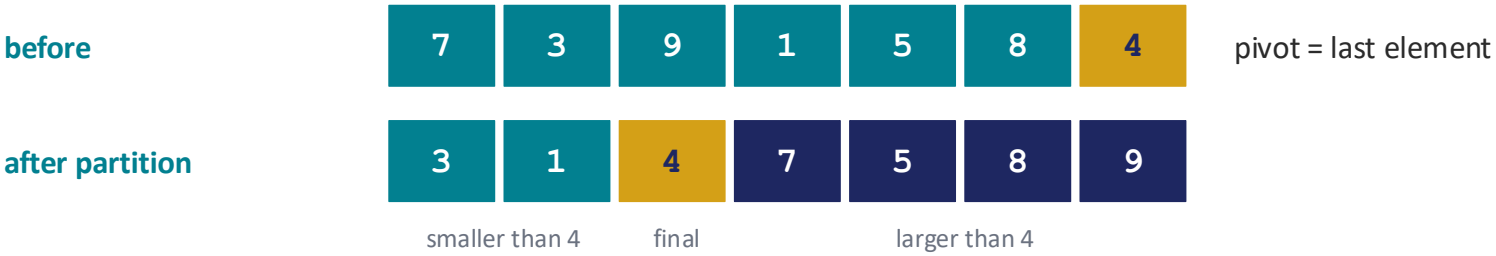
Idea pick a pivot, put smaller values on its left and larger on its right, repeat on both sides

Dry run partition and recursion traced on 7 3 9 1 5 8 4

Complexity O(n log n) on average, O(n^2) in the worst case, verified

The idea: pick a pivot and split the array

Choose one value as the pivot. Rearrange so smaller values are on its left and larger values on its right. The pivot is now in its FINAL place. Repeat the same steps on the left part and on the right part.



After each partition the pivot is final and the two sides never need to be compared with each other again, so each side is sorted on its own, in the same way.

The algorithm, step by step

1. quickSort(a, lo, hi): if lo >= hi stop. Zero or one element is already sorted.
2. Choose the pivot: here the LAST element, pivot = a[hi].
3. Scan j from lo to hi - 1. Whenever a[j] < pivot, grow the small zone (i++) and swap a[j] into it.
4. Swap the pivot into the gap just after the small zone. It is now final, at index p = i + 1.
5. Call quickSort on lo..p-1 and on p+1..hi. The pivot is left alone.

Partition invariant
While j scans, a[lo..i] holds values smaller than the pivot and a[i+1..j-1] holds values that are not smaller. Index i marks the end of the small zone.

Quick sort in C

```
int partition(int a[], int lo, int hi)
{
    int pivot = a[hi];
    int i = lo - 1;     /* small zone end */

    for (int j = lo; j < hi; j++)
    {
        if (a[j] < pivot)
        {
            i++;
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[i + 1], &a[hi]);  /* pivot home */
    return i + 1;
}
```

```
void quickSort(int a[], int lo, int hi)
{
    if (lo < hi)
    {
        int p = partition(a, lo, hi);
        quickSort(a, lo, p - 1);
        quickSort(a, p + 1, hi);
    }
}
```

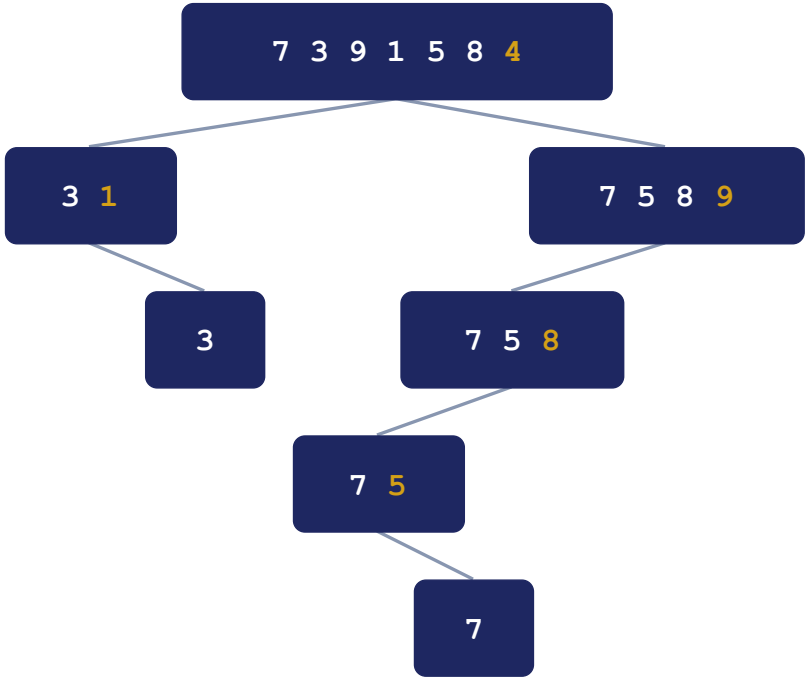
- Pivot = a[hi], the last element.
- swap(&x, &y) exchanges two ints through a temporary variable.
- The pivot is not passed to either recursive call: it is already final.

One partition on 7 3 9 1 5 8 4 (verified)

Step	Question	Action	Array after the step	i
start	pivot = a[6] = 4	i = -1	7 3 9 1 5 8 4	-1
j = 0	7 < 4 ? no	skip	7 3 9 1 5 8 4	-1
j = 1	3 < 4 ? yes	i = 0, swap a[0], a[1]	3 7 9 1 5 8 4	0
j = 2	9 < 4 ? no	skip	3 7 9 1 5 8 4	0
j = 3	1 < 4 ? yes	i = 1, swap a[1], a[3]	3 1 9 7 5 8 4	1
j = 4	5 < 4 ? no	skip	3 1 9 7 5 8 4	1
j = 5	8 < 4 ? no	skip	3 1 9 7 5 8 4	1
end	pivot home	swap a[i+1], a[6]	3 1 4 7 5 8 9	p = 2

6 comparisons, one per j. The pivot 4 is final at index 2, with 3 1 on its left and 7 5 8 9 on its right.

The recursion tree for 7 3 9 1 5 8 4



Comparisons per call

7 values: 6

3 1: 1

7 5 8 9: 3

7 5 8: 2

7 5: 1

Total: 13

Gold = pivot of that call. Each box is shown before its partition, with the pivot last.

Best, worst and average case, measured (n = 8)

Case	Input	Comparisons
Best	`1 2 4 3 7 6 8 5  (pivot lands mid-way each time)`	13
Typical	`5 2 8 1 9 3 7 4`	16
Average	`mean over all 40320 orderings`	16.92
Worst	`1 2 3 4 5 6 7 8  (already sorted)`	28
Worst	`8 7 6 5 4 3 2 1  (reversed)`	28

- Over all 40320 orderings of eight values the comparisons range from 13 to 28.
- Sorted input is the WORST case here: the last element is always the largest, so each partition removes only the pivot. Insertion sort needs just 7 comparisons on the same input.
- Best case: the pivot splits the array in the middle every time, giving O(n log n).

Growth: sorted input hurts, typical input does not

n	sorted input	growth	pseudo-random input	growth
100	4950	-	667	-
200	19900	4.02 times	1517	2.27 times
400	79800	4.01 times	3749	2.47 times
800	319600	4.01 times	7871	2.10 times

Sorted input
Exactly n(n-1)/2 comparisons. Doubling n multiplies the cost by about 4: O(n^2).

Pseudo-random input
Doubling n costs only a little more than double: the O(n log n) pattern. One fixed array per size.

Fixing the worst case: median-of-three pivot

sorted input, n	last-element pivot: comparisons	max depth	median-of-three: comparisons	max depth
100	4950	100	480	7
400	79800	400	2698	9
1000	499500	1000	7987	10

Median-of-three
Take the middle value of a[lo], a[mid] and a[hi] as the pivot. On sorted data it is the true median, so every split is even.

Why depth matters
Each nested call uses stack memory. 1000 nested calls for 1000 sorted values is a risk for big arrays; 10 is not.

Properties of quick sort

Property	Answer	Evidence
Stable	No	Marks Asha 60, Bina 60, Charu 75, Dev 80 sort to Bina, Asha, Charu, Dev
In-place	Yes, plus stack	No second array; depth 23 for 1000 random values, 1000 for 1000 sorted
Adaptive	No	Sorted input is the worst case: 28 comparisons for n = 8
Time: best / average / worst	O(n log n) / O(n log n) / O(n^2)	Measured on the previous slides

Mixed input, n = 8 (5 2 8 1 9 3 7 4). Comparisons: quick sort 16, insertion sort 18, bubble sort with the flag 25. Quick sort wins on typical data but loses its edge on sorted data, and unlike insertion and bubble sort it is not stable.

Pivot, partition, recurse

- Partition puts smaller values left and larger right; the pivot is then final. Recurse on both sides.
- Dry run on 7 3 9 1 5 8 4: the first partition takes 6 comparisons, the whole sort 13.
- Average O(n log n) (16.92 comparisons for n = 8 over all orderings), worst case O(n^2) on sorted input (28).
- In-place but not stable and not adaptive. Median-of-three fixes sorted input: 7987 comparisons for n = 1000 instead of 499500.

EXIT TICKET Partition 8 3 7 1 5 with the last element as pivot. Show the array after the partition, the pivot's final index and the number of comparisons.